

NAME ; ADITYA KUMAR

REG NO : 2647203

QUESTTION 1

Code :

```
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <ctype.h>
#include <stdlib.h>

int bin_ser(int ele, int *arr, int n){
    int st = 0, en = n - 1;
    while (st <= en){
        int mid = (st +(en-st)/2);
        if (arr[mid] == ele){
            return mid;
        }
        else if (arr[mid] < ele){
            st = mid + 1;
        }
        else if (arr[mid] > ele){
            en = mid - 1;
        }
    }
}
```

```

        else{
            printf("ERROR");
            break;
        }
    }
    return -1;
}

int main(){
    int arr[100];
    int a,n;

    printf("Enter size of array :");
    scanf("%d",&n);

    printf("enter the elements (in sorted order) :\n");
    for (int i = 0; i<n; i++){
        scanf("%d", &arr[i]);
    }

    printf("Enter element to search ;");
    scanf("%d",&a);

    int result = bin_ser(a, arr, n);

    if (result != -1){
        printf("Element found at index %d\n", result);
    }
}

```

```

    }

    else{

        printf("-1 \n");

    }

    return 0;

}

```

Output:

```

● PS E:\9. Projects\0. Github\MCA DSA Lab + CIA\CIA1> cd "e:\9. Projects\0. Github\MCA DSA Lab + CIA\CIA1\" ; if ($?) { gcc Q1.c -o Q1 } ; if ($?) { .\Q1 }
Enter size of array :5
enter the elements (in sorted order) :
1
2
3
3
5
Enter element to search ;3
Element found at index 2
● PS E:\9. Projects\0. Github\MCA DSA Lab + CIA\CIA1> cd "e:\9. Projects\0. Github\MCA DSA Lab + CIA\CIA1\" ; if ($?) { gcc Q1.c -o Q1 } ; if ($?) { .\Q1 }
Enter size of array :5
enter the elements (in sorted order) :
1
2
3
5
9
Enter element to search ;6
-1

```

```

Enter size of array :5
enter the elements (in sorted order) :
1
2
3
3
5
Enter element to search ;3
Element found at index 2

```

```

Enter size of array :5
enter the elements (in sorted order) :
1
2
3
5
9
Enter element to search ;6
-1

```

Question 2

```
#include <stdio.h>
#include <stdlib.h>
```

```
typedef struct Node {
    int data;
    struct Node* next;
} Node;
```

```
Node* head = NULL;
Node* tail = NULL;
```

```
void insertAtEnd(int value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = value;
    newNode->next = NULL;
    if (head == NULL) {
        head = newNode;
        tail = newNode;
    } else {
        tail->next = newNode;
        tail = newNode;
    }
}
```

```
void deleteFirstNode() {
```

```

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    Node* temp = head;
    int value = temp->data;
    head = temp->next;
    if (head == NULL) {
        tail = NULL;
    }
    free(temp);
    printf("Deleted value: %d\n", value);
}

```

```

void displayList() {
    Node* curr = head;
    while (curr != NULL) {
        printf("%d -> ", curr->data);
        curr = curr->next;
    }
    printf("NULL\n");
}

```

```

void reverseList() {
    Node* prev = NULL;
    Node* current = head;
    Node* next = NULL;

```

```

tail = head;
while (current != NULL) {
    next = current->next;
    current->next = prev;
    prev = current;
    current = next;
}
head = prev;
}

int main() {
    int choice, value;
    while (1) {
        printf("1. Insert at end\n");
        printf("2. Delete first node\n");
        printf("3. Display list\n");
        printf("4. Reverse list\n");
        printf("5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter value to insert: ");
                scanf("%d", &value);
                insertAtEnd(value);
                break;
            case 2:

```

```
        deleteFirstNode();
        break;
    case 3:
        displayList();
        break;
    case 4:
        reverseList();
        break;
    case 5:
        exit(0);
    default:
        printf("Invalid choice\n");
    }
}
return 0;
}
```

Code:

Output:

```
1. Insert at end
2. Delete first node
3. Display list
4. Reverse list
5. Exit
Enter choice: 1
Enter value to insert: 6
1. Insert at end
2. Delete first node
3. Display list
4. Reverse list
5. Exit
Enter choice: 1
Enter value to insert: 5
1. Insert at end
2. Delete first node
3. Display list
4. Reverse list
5. Exit
Enter choice: 3
5 -> 6 -> 5 -> NULL
```

```
1. Insert at end
2. Delete first node
3. Display list
4. Reverse list
5. Exit
Enter choice: 2
Deleted value: 4
```

```
1. Insert at end
2. Delete first node
3. Display list
4. Reverse list
5. Exit
Enter choice: 3
5 -> 6 -> 5 -> NULL
```

```
Enter choice: 4
1. Insert at end
2. Delete first node
3. Display list
4. Reverse list
5. Exit
Enter choice: 3
5 -> 6 -> 5 -> NULL
1. Insert at end
2. Delete first node
3. Display list
4. Reverse list
5. Exit
Enter choice: █
```

Question 3

Code:

```
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <ctype.h>
#include <stdlib.h>

typedef struct Stack{
    char arr[100];
    int size;
    int top;
} Stack;

int main(){
```

```

int n;

char arr[100];


printf("Enter the size of the stack :");
scanf("%d",&n);


printf("Enter the brackets : \n");
for(int i= 0 ; i < n; i++){
    scanf(" %c", &arr[i]);
}


Stack *st = (Stack *)malloc(sizeof(Stack));
st->top = -1;
st->size = n;


int isBalanced = 1;
for (int i=0 ; i< n; i++){
    if (arr[i] == '(' || arr[i] == '[' || arr[i] == '{'){
        st->arr[++st->top] = arr[i];
    }
    else if (arr[i] == ')' || arr[i] == ']' || arr[i] == '}{
        if (st->top == -1 ||
            (arr[i] == ')' && st->arr[st->top] != '(') ||
            (arr[i] == ']' && st->arr[st->top] != '[') ||
            (arr[i] == '}' && st->arr[st->top] != '{')){
            isBalanced = 0;
            break;

```

```

        }
        st->top--;
    }
}

if (st->top != -1){
    isBalanced = 0;
}

if (isBalanced){
    printf("\n Brackets are Balanced");
}
else{
    printf("\n Brackets are NOT Balanced");
}

free(st);
return 0;
}

```

Output :

Enter the size of the stack :5

Enter the brackets :

[
[
[
]
]

Brackets are NOT Balanced

PS E:\9. Projects\0. GitHub\MCA DSA Lab + CIA\CIA1> 4

4

PS E:\9. Projects\0. GitHub\MCA DSA Lab + CIA\CIA1> cd "e

Enter the size of the stack :4

Enter the brackets :

[

{

}
]

Brackets are Balanced

PS E:\9. Projects\0. GitHub\MCA DSA Lab + CIA\CIA1> █